

Data on the Line · Hazırlık notu

Talep tahmini ve hat dengeleme · okuma süresi yaklaşık 20 dakika

Bu not, vakada kullanacağınız kavramları vakadan bağımsız küçük örneklerle anlatır. Örneklerdeki sayıların vakayla bir ilgisi yoktur. Oturumdan önce okumanız ve kurulum testini çalıştırmanız yeterli.

1. Takt süresi

Takt süresi, talebe yetişmek için bir ürünün hattan kaç saniyede bir çıkması gerektiğini gösterir:

$$\text{takt süresi} = \text{kullanılabilir süre} / \text{üretilmesi gereken adet}$$

Kullanılabilir süre, vardiya süresinden molalar, planlı bakım ve duruşlar düşüldükten sonra kalan süredir. Örneğin günde 50.400 saniye net çalışma süresi varsa ve 600 adet üretilmesi gerekiyorsa takt süresi 84 saniyedir. Bir istasyonun yükü 84 saniyeyi aşıyorsa hat bu tempoya yetişemez.

Yalın üretim literatüründe takt, yalnızca müşteri talebinden hesaplanır. Üretim adedine stok tamamlama ya da hurdaya ayrılan ürünlerin yeniden üretimi de eklendiğinde bulunan değer, aslında **planlı çevrim süresidir**.

Uygulamada ikisine de çoğu zaman "takt" denir.

2. Montaj hattı dengeleme

Bir montaj hattında ürün, sırayla istasyonlardan geçer ve her istasyonda bazı işler yapılır. **Hat dengeleme**, işleri istasyonlara dağıtma problemidir. Temel kavramlar:

- **İş (görev):** bölünemeyen en küçük çalışma birimi; her işin bir süresi vardır.
- **Öncelik ilişkisi:** bazı işler başka işler bitmeden başlayamaz. Bir iş, kendisinden önce yapılması gereken bir işten daha önceki bir istasyona konamaz.
- **İstasyon yükü:** bir istasyona atanan işlerin süreleri toplamı.
- **Darboğaz:** yükü en yüksek istasyon. Hattın gerçek temposunu bu istasyon belirler; bir hat, en yoğun istasyonu kadar hızlı çalışabilir.

Literatürde iki temel problem tipi vardır:

- **Tip 1:** takt süresi verilir; amaç, hiçbir istasyonun yükü takt süresini aşmadan **istasyon sayısını en aza indirmektir**.
- **Tip 2:** istasyon sayısı verilir; amaç, **en yoğun istasyonun yükünü**, yani hattın çevrim süresini **en aza indirmektir**.

İstasyon sayısı için bir **alt sınır** hesaplanabilir:

$$\text{alt sınır} = \lceil \text{toplam iş içeriği} / \text{takt süresi} \rceil$$

Örneğin toplam 410 saniyelik iş ve 84 saniyelik takt için $410 / 84 = 4,88$ olduğundan en az 5 istasyon gerekir. Alt sınır, daha az istasyonla hattın kurulamayacağını kanıtlar; ama bu kadar istasyonun yeteceğini garanti etmez. İşler bölünemediği ve öncelik ilişkileriyle başka kurallar da devreye girdiği için daha fazla istasyon gerekebilir.

Dengenin ne kadar iyi olduğunu **hat verimliliğiyle** ölçebilirsiniz:

$$\text{hat verimliliği} = \text{toplam iş içeriği} / (\text{istasyon sayısı} \times \text{takt süresi})$$

Verimlilik %100'e ne kadar yakınsa istasyonlardaki boş süre o kadar azdır.

Karma modeli hatlar. Aynı hatta birden fazla ürün üretiliyorsa işlerin süresi ürüne göre değişir. Bu durumda her iş için, ürün karmasına göre **ağırlıklı ortalama süre** kullanılır: bir ürün üretimin %40'ını oluşturuyorsa o ürünün

süresi ortalamaya %40 ağırlıkla girer. Ancak ortalamada takt süresine sığan bir istasyon, tek bir ürün geldiğinde takt süresini epey aşabilir. Gerçek hatlarda bu kısa süreli aşımalar ara stoklar ve ürün sıralamasıyla karşılanır; ama ne kadar aşımın kabul edilebileceği ayrıca kontrol edilmelidir.

3. Doğrusal tamsayılı programlama (MILP)

Hat dengeleme problemleri genellikle **doğrusal tamsayılı programlama** (mixed-integer linear programming, MILP) ile çözülür. Bir MILP modeli üç parçadan oluşur:

- **Karar değişkenleri:** örneğin "i işi k istasyonunda mı?" sorusunun cevabı olan 0–1 (ikili) değişkenler.
- **Amaç fonksiyonu:** en küçüklenecek ya da en büyüklenecek tek bir ifade.
- **Kısıtlar:** çözümün uyması gereken kurallar.

"Doğrusal", amaç fonksiyonunun ve kısıtların karar değişkenlerinin sabit katsayılarla çarpılıp toplanmasından oluştuğu anlamına gelir. İki değişkeni birbiriyle çarpmak ya da bir değişkenin karesini almak doğrusal değildir.

İki yararlı modelleme tekniği:

- **En büyük değeri en küçükleme:** "en yoğun istasyonun yükü" doğrudan doğrusal bir ifade değildir. Bunun yerine yardımcı bir değişken tanımlanır, her istasyonun yükünün bu değişkeni aşamayacağı kısıtı eklenir ve bu değişken en küçüklendir.
- **Sıralı iki amaç:** iki hedef kesin bir öncelik sırasıyla isteniyorsa, ikisi tek bir amaç fonksiyonunda birleştirilebilir. Birinci hedefe öyle büyük bir ağırlık verilir ki ikinci hedefteki hiçbir iyileşme birinci hedefteki tek bir birimlik kötüleşmeyi telafi edemez. Birinci hedef tamsayı değerler alıyorsa ve ikinci hedef negatif olamıyorsa, ağırlığın ikinci hedefin alabileceği en büyük değerden büyük olması yeterlidir.

Bir MILP çözücüsü, yeterli süre verildiğinde en iyi çözümü bulur ve en iyi olduğunu kanıtlar. **Sezgisel yöntemler** ise hızlıdır ama buldukları çözümün en iyi olduğunu garanti etmez.

Aşağıdaki örneğin hat dengelemeyle ilgisi yok: üç iş iki makineye dağıtılıyor ve daha yoğun makinenin yükü en küçüklenecek. Yine de vakada ihtiyaç duyacağınız bütün PuLP yapılarını içeriyor: iki kümeyle indekslenen 0–1 değişkenler, toplamlar, bir amaç fonksiyonu, kısıtlar ve yukarıdaki yardımcı değişken tekniği.

```
import pulp

JOBS, MACHINES = ["A", "B", "C"], [1, 2]
HOURS = {"A": 4, "B": 3, "C": 2}

m = pulp.LpProblem("ornek", pulp.LpMinimize)
# x[j][k] = 1: j işi k makinesinde
x = pulp.LpVariable.dicts("x", (JOBS, MACHINES), cat="Binary")
# daha yoğun makinenin yükü (saat)
longest = pulp.LpVariable("longest", lowBound=0)

# amaç: yoğun makinenin yükünü en aza indir
m += longest
# her iş tam olarak bir makinede
for j in JOBS:
    m += pulp.lpSum(x[j][k] for k in MACHINES) == 1
# hiçbir makinenin yükü longest'i aşamaz
for k in MACHINES:
    m += pulp.lpSum(HOURS[j] * x[j][k] for j in JOBS) <= longest

m.solve(pulp.PULP_CBC_CMD(msg=False))
plan = {j: k for j in JOBS for k in MACHINES if x[j][k].value() > 0.5}
print(pulp.LpStatus[m.status], plan)
```

Örnek PuLP'un 3. sürümüyle yazıldı. Colab'de hazır gelen 4. sürümde bu yazım değişti; bu yüzden Colab'de önce ayrı bir hücrede `!pip install -q "pulp>=2.7,<4"` çalıştırın. Kendi Mac'inizde "Bad CPU type" gibi bir hata alırsanız `pip install highspy` ile HiGHS çözücüsünü kurun ve `m.solve(...)` satırında

`pulp.PULP_CBC_CMD(msg=False)` yerine `pulp.HiGHS(msg=False)` yazın. Vaka notebook'u bu iki adımı sizin yerinize otomatik olarak yapıyor.

4. Tahmin hatasını ölçmek

İki ölçüt birlikte kullanılır:

$$\text{WAPE} = \sum | \text{gerçekleşen} - \text{tahmin} | / \sum \text{gerçekleşen}$$

$$\text{Bias} = \sum (\text{tahmin} - \text{gerçekleşen}) / \sum \text{gerçekleşen}$$

WAPE, toplam talebin yüzdesi olarak ne kadar yanıldığınızı gösterir. Bias ise hatanın yönünü gösterir: pozitifse sistematik olarak fazla, negatifse sistematik olarak eksik tahmin ediyorsunuz demektir.

Örnek: üç ayın gerçekleşen talebi 100, 120 ve 80, tahminler 110, 110 ve 90 olsun. Mutlak hataların toplamı $10 + 10 + 10 = 30$, toplam talep 300 olduğu için WAPE %10'dur. Hataların işaretli toplamı $+10 - 10 + 10 = +10$ olduğu için bias +%3,3'tür. İkisini birlikte okuyun: WAPE'si düşük ama hep aynı yönde yanılan bir modelin de sorunu vardır.

5. Bir tahmin modelini sınamak

Bir modelin işe yarayıp yaramadığı, eğitildiği veride değil, **görmediği veride** ne kadar iyi tahmin yaptığına bakılarak anlaşılır. Bu yüzden veri bir eğitim ve bir test kısmına ayrılır: model eğitim kısmıyla kurulur, test kısmında değerlendirilir. Bu ayrımın nasıl yapılacağı, verinin yapısına ve modelin gerçekte nasıl kullanılacağına uygun olmalıdır.

Her model, basit bir **kıyas yöntemiyle** karşılaştırılmalıdır. Örneğin "geçen yılın aynı ayı" tahmini (mevsimsel naif yöntem) bu iş için sıkça kullanılır. Bu kıyas yöntemini geçemeyen bir model, ne kadar karmaşık olursa olsun kullanılmaya değmez.